



INTRODUCTION TO ADVANCED ROBOTICS

PROJECT 1

SEEING SHAPES

WHAT'S NEXT?

In Module 1 you built a program that could tell red from yellow. That worked because every colour has a number attached to it, so the computer could look up a range and make a decision.

This project asks a much harder question. Nobody tells the computer what colour a block is. It has to work out the shape from the picture alone — triangle, square, circle or rectangle — and then put a name on screen.

The starter file already contains all of the difficult mathematics. Your job is to supply the missing **measurements** and the **decision** that turns them into a **single word**.
Two windows will open when the program runs: `Shape Detection` with your label drawn on it, and `Threshold View (Binary)` with the cleaned-up black and white picture the computer actually sees. Keep both open. Most of your debugging happens in the second one.

FROM COLOUR TO SHAPE

Colour detection asks a question about a pixel: which numbers fall inside a range? Shape detection asks a question about an outline: how far around is it, and how much of its own box does it fill?

That means hue, saturation and value are no longer needed at all. We throw the colour away on purpose and keep only light and dark.

This is the first project where OpenCV does the seeing and you do the deciding. Every number that ends up on screen is a number you wrote.

THE PIPELINE

Five lines turn a camera frame into a list of candidate shapes. You will not change any of these five lines — they are already written for you — but you need to know what each one hands to the next.

- 1 Greyscale**
One brightness number per pixel instead of three colour numbers.
- 2 Blur**
Smooths sensor noise so the outline is not full of holes.
- 3 Threshold**
Turns everything darker than 110 white, everything else black.
- 4 Close**
Darns small holes so one shape stays one shape.
- 5 Contours**
Traces each outline and hands you a list of them.

GREYSCALE

A camera hands you a three-number pixel: blue, green, red. For shape work that is three times more data than you need, and the colour values only add noise.

`cvtColor` converts the frame to one brightness number per pixel. White stays 255, black stays 0, and everything in between keeps its place on the grey scale.

```
img_h, img_w = img.shape[:2]  
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Two things to notice. `img.shape[:2]` gives you height and width only — the trailing 3, which is the colour channels, is chopped off by the `:2`. And `BGR2GRAY` is not a colour choice: it is the channel order OpenCV uses, which is the reverse of the `RGB` you are used to.

BLUR

The camera's sensor is never perfect. A single dark pixel of noise becomes a hole in your shape, and a hole becomes a broken contour.

A Gaussian blur replaces every pixel with the average of itself and its neighbours. That pushes isolated noise down while leaving large solid regions alone.

```
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

The `(5, 5)` is a window size: five pixels across, five pixels down. Bigger numbers smooth more. The `0` at the end means 'work out the standard deviation for me', and it is the only value you will ever need.

THRESHOLD

Now you make a decision about every single pixel: is it dark enough to be part of a shape, or not?

`THRESH_BINARY_INV` is the important half. 'INV' means inverted — the dark pixels become white and the pale background becomes black. That is deliberate. We want white shapes on a black background, because that is what contour finding expects.

```
_ , thresh = cv2.threshold(blurred, 110, 255, cv2.THRESH_BINARY_INV)
```

`110` is the only number on this slide you will ever be asked to change. Turn it up and pale shapes disappear; turn it down and every shadow on the table becomes a shape. If a shape refuses to detect, this number is the first thing to touch.

The `_` on the left is a variable you are deliberately throwing away. OpenCV returns the threshold value as well as the image, and this program has no use for it.

CLOSING THE OUTLINE

Blur helps, but it never quite finishes the job. The outline is still broken in places, and a broken outline is detected as two smaller shapes instead of one.

A morphological close swells the outline by a few pixels and then shrinks it back. The net effect is that small gaps vanish while the overall size stays the same.

```
kernel = np.ones((5, 5), np.uint8)
thresh_clean = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)
```

`np.ones((5, 5), np.uint8)` builds a 5×5 block of ones, and that block is the brush. Swapping `MORPH_CLOSE` for `MORPH_OPEN` would do the opposite job: remove specks instead of gaps.

CONTOURS

This is the step that actually produces the list you are about to measure.

`findContours` walks along every white shape and returns the pixels on its border, in order, as a list of points. Each shape comes back as one NumPy array of coordinates.

```
contours, _ = cv2.findContours(  
    thresh_clean, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE  
)
```

Two flags are doing real work here. `RETR_EXTERNAL` keeps only the outermost outline, so a shape drawn inside another shape is not reported twice. `CHAIN_APPROX_SIMPLE` throws away points sitting in a straight line between two others — fewer numbers to store, and a square arrives as four corners instead of a hundred.

The `_` on the left of the equals sign is OpenCV's hierarchy — a map of which shape sits inside which. This program has no use for it, so it goes straight in the bin.

THREE MEASUREMENTS

Everything that follows is built from three numbers. Take them one at a time, and give each one its own guard.

area — `cv2.contourArea(cnt)`

How many pixels the shape covers. This is also how you throw away noise.

perimeter — `cv2.arcLength(cnt, True)`

The distance all the way around the outline. The True means the contour is closed.

bounding box — `cv2.minAreaRect(cnt)`

The smallest rectangle that still contains the shape, rotated to hug it.

```
area = cv2.contourArea(cnt)
peri = cv2.arcLength(cnt, True)
rect = cv2.minAreaRect(cnt)
(cx, cy), (w, h), angle = rect
rect_area = w * h
```

Notice the fourth line. `(cx, cy), (w, h), angle = rect` is Python unpacking: one returned value, immediately split into three differently named ones. You will use this pattern constantly.

THREE METRICS

Raw measurements are awkward to compare. A circle of 3000 pixels and a square of 3000 pixels are the same size and nothing else. These three ratios turn size into shape.

```
extent = float(area) / rect_area
circularity = (4 * np.pi * area) / (peri ** 2)
aspect_ratio = float(min(w, h)) / max(w, h) if max(w, h) > 0 else 0
```

Two Python ideas in three lines. `**` is the power operator, so `peri ** 2` squares the perimeter. The third line ends in `... if ... else 0`, a conditional expression: divide only when the larger side is not zero, otherwise fall back to zero.

Metric	Formula	Triangle	Square	Circle	Rectangle
extent	$\text{area} / \text{rect_area}$	~0.50	~1.00	~0.79	~1.00
circularity	$4\pi \cdot \text{area} / \text{peri}^2$	~0.60	~0.79	~1.00	~0.70
aspect_ratio	$\min(w, h) / \max(w, h)$	~0.87	~1.00	~1.00	~0.50

Extent fills how much of its own box the shape takes up. **Circularity** scores how close the outline is to a perfect circle. **Aspect ratio** compares the sides: 1.0 means a square.

COUNTING CORNERS

Extent and circularity are not enough on their own. A square and a rectangle both fill their box almost completely, so you need one more clue: how many corners has this shape got?

`approxPolyDP` simplifies the outline, discarding points that do not change the direction much, and hands back the corners that are left.

```
approx = cv2.approxPolyDP(cnt, 0.04 * peri, True)
num_vertices = len(approx)
```

`0.04 * peri` is the tolerance, and it scales with the shape — a big shape is allowed to be sloppy, a small one is not. Set it to 0 and you get every original pixel back; set it far too high and a square collapses to a single line.

- *These two lines are already written for you in the starter file. Do not retype them — just notice that `num_vertices` is the number you will test in a moment.*

GATES AND GUARDS

Before a shape is measured, the program throws most contours away. Before a shape is divided, it checks the divisor is not zero. Both habits are worth copying.

1 The area gate

A shape has to occupy between 2% and 80% of the cropped view. Anything smaller is dust or a shadow; anything larger is your hand, or the rim of the plate.

2 The zero guards

`peri == 0` and `rect_area == 0` look fussy, but dividing by zero raises `ZeroDivisionError` and kills the whole program. A camera pointed at nothing will do this to you within a minute.

```
if area < (img_h * img_w * 0.02) or area > (img_h * img_w * 0.80):  
    continue
```

The `or` matters. Either condition on its own is enough to reject the contour, so the line is long and hard to read on purpose — split it across lines with brackets if that helps you.

THE DECISION

Now the payoff. Four tests, tried in order, and the first one that succeeds names the shape.

```
object_type = "Unknown"

if extent < 0.65 or num_vertices == 3:
    object_type = "Triangle"
elif extent >= 0.82 and aspect_ratio >= 0.78:
    object_type = "Square"
elif circularity >= 0.85 or (0.68 <= extent < 0.82 and aspect_ratio >= 0.80):
    object_type = "Circle"
else:
    object_type = "Rectangle"
```

Starting at "Unknown" is not decoration. It guarantees the variable exists before any test runs, so nothing downstream can trip over an undefined name. If you later make the thresholds aggressive enough that every test fails, `Unknown` is what appears on screen instead of a crash.

Read the branches as English: 'if it fills less than 65% of its box, or it has three corners, it is a triangle. Otherwise, if it fills at least 82% and its sides are nearly equal, it is a square.' That is the whole trick — the numbers are just the shape of the sentence.

WHY ORDER MATTERS

Look at the square test and the circle test again. Both ask for `aspect_ratio >= 0.78`, and a perfect circle has an aspect ratio of 1.00. A circle would satisfy the square test if the square test ran second.

This is why the chain is an `if / elif / elif / else` and not four independent `if` statements. Python runs the tests top to bottom and stops at the first match, so the order you write is part of the meaning.

It is also why the triangle test comes first. A triangle fills only half of its own box, so `extent < 0.65` claims it before any other test has a chance to guess.

- *Before you write the chain, predict the order yourself. For each of the four shapes, look at the table on the Three metrics slide and decide which test should claim it. Then check your answer against the code.*

STEP 1

On your computer, open VS Code, open your project folder, and open the file labelled **M2-P1-Starter.py**.

You will see sections marked `# TODO:` with the code missing between the `# ---` lines. There are eleven of them, and they run from the imports at the top of the file to the camera setup near the bottom.

Before you fill in any of them, send the file across and run it. Seeing it fail first tells you exactly which errors belong to the missing code.

Turn the arm on and give it about two minutes to boot, then run this in the VS Code terminal.

```
scp M2-P1-Starter.py er@192.168.1.149:Documents
```

STEP 2

Open Real VNC Viewer and connect to the Arm - Pump device. Open 'er's Home', then the Documents folder on the left panel.

Right-click the empty space in the folder, choose `Open in Terminal`, and run the starter file.

```
python M2-P1-Starter.py
```

You will not get as far as the camera, and the two failures are worth reading carefully, because they are different kinds of failure.

The robot section sits inside a `try / except` block, so the missing `MyCobot280` import does not stop the program — it prints a hardware warning and carries on. The missing `cv2` import is not

```
Robot hardware warning: name 'MyCobot280' is not defined  
Continuing with OpenCV camera pipeline only...
```

```
NameError: name 'cv2' is not defined
```

Neither is a broken camera. `try / except` lets the problem be reported, then ignored — the message just tells you which import is missing.

STEP 3

Fill in the first TODO at the very top of the file. Four imports, and each one earns its place:

`cv2`

OpenCV — every `cv2.` call in this project comes from here.

`numpy as np`

NumPy under the alias `np`, used for arrays and math in the shape code.

`time`

The standard library. `time.sleep()` paces the program against the arm.

`MyCobot280`

The arm driver from `pymycobot`. Written specifically for this course.

```
import cv2
import numpy as np
import time

from pymycobot.mycobot280 import MyCobot280
```

The `as np` is an alias: `np` is a short nickname for `numpy`. Keep the blank line — it separates other people's code from yours.

STEP 4

The next TODO sits inside the `try` block. This is the code that wakes the arm up.

`MyCobot280('/dev/ttyAMA0', 1000000)` takes two arguments: which serial port the arm is plugged into, and how fast to talk to it. On another machine the port may be `ttyUSB0`, so if this line throws, check the port before you suspect the arm.

Each `sleep` gives the command before it time to finish. Remove them and the arm misses instructions — which looks exactly like a broken arm.

```
print("Connecting to myCobot280...")

mc = MyCobot280('/dev/ttyAMA0', 1000000)
time.sleep(0.5)
mc.power_on()
time.sleep(0.5)
```

`mc` is the name you give the arm. Every later line that talks to the hardware goes through it, so you only ever type the port once.

STEP 5

One TODO left in this section: park the arm somewhere safe before the camera work starts.

Six numbers, one per joint, in degrees. Zero means ‘no bend’, and the signs say which way. Folding the arm up and out of shot is deliberate — a metal arm parked in view is a shape your program will try to measure.

The 50 is how fast to travel there.

```
home_pos = [0, 45, -90, -45, 0, 0]
mc.send_angles(home_pos, 50)
time.sleep(2.0)
```

This is a `list`, not a `tuple`. The `[ ]` brackets mean you can change it later, which is exactly what you will want when you go on to record positions for the sorting bins.

STEP 6

Send the file across again and run it. This time all four imports exist, so the robot should wake up and two windows should open.

```
scp M2-P1-Starter.py er@192.168.1.149:Documents
```

```
python M2-P1-Starter.py
```

The status line will say `Searching...` and stay there. Put a shape on the plate and nothing happens.

That is expected, and it is worth pausing on. Five stages of image processing are running correctly — greyscale, blur, threshold, close and the contour search have all executed. What is missing is the part that **measures**.

Read the two windows before you change anything. In `Threshold View (Binary)` your shape should already be a clean white blob on black. If it is not, the problem is upstream of the TODOs and no amount of typing will help. Learn to look at the boring picture first.

STEP 7

First measurement TODO. How much of the view does the shape cover, and is it worth measuring at all?

```
area = cv2.contourArea(cnt)
if area < (img_h * img_w * 0.02) or area > (img_h * img_w * 0.80):
    continue
```

`img_h * img_w` is the total pixel count of the cropped view, so the gate is expressed as a percentage and survives you changing the camera resolution later.

`continue` does not mean 'skip this one line'. It abandons the whole loop iteration and moves on to the next contour. Forgetting that is how people end up measuring a dust speck.

STEP 8

Second measurement. How far is it all the way around?

```
peri = cv2.arcLength(cnt, True)
if peri == 0:
    continue
```

The `True` asks for a closed outline — join the last point back to the first. Leave it off and you lose one edge in three on a triangle, which is more than enough to make the circularity figure wrong without looking obviously broken.

The guard underneath looks pedantic, but remember that this perimeter is about to become a divisor. Guard the divisor, not the symptom.

STEP 9

Third measurement, and the one that needs unpacking.

```
rect = cv2.minAreaRect(cnt)
(cx, cy), (w, h), angle = rect
rect_area = w * h
if rect_area == 0:
    continue
```

`minAreaRect` does not give you an upright box. It rotates one to hug the shape, so a shape leaning at 30° still gets a tight fit. That is why it hands back an `angle`, and why you get `(w, h)` rather than `(left, top, right, bottom)`.

Read the second line right to left: the centre arrives first, then the size, then the angle. And note that `cx` and `cy` are never used by the decision — they are where the label gets drawn. Not every value you are handed has to be used.

STEP 10

Now turn the raw measurements into the three ratios. These three lines are what the whole project turns on.

```
extent = float(area) / rect_area
circularity = (4 * np.pi * area) / (peri ** 2)
aspect_ratio = float(min(w, h)) / max(w, h) if max(w, h) > 0 else 0
```

Two Python ideas in three lines. `**` is the power operator, so `peri ** 2` squares the perimeter. And the third line ends in `if ... else 0`, a conditional expression written out longhand it would be four lines with an `if` and a `pass`.

- *Type these three lines exactly as they appear. A single missing bracket here will not be reported until the program is already running.*

STEP 11

One line, and it is not filler.

```
object_type = "Unknown"
```

This guarantees the variable exists before any test runs, so nothing downstream can trip over an undefined name. In this project the `else` branch always overwrites it — but tune the thresholds hard enough that every test fails, and `Unknown` is what appears on screen instead of a

- *Set the safe default first, then overwrite it. This one habit prevents an entire class of beginner bug.*

In Python a name has to exist before it is used, and reading an undefined name is an error, not an empty string. Declaring it up front is cheaper than debugging that later.

STEP 12

The last big TODO. Four tests in order; the first match names the shape.

```
object_type = "Unknown"

if extent < 0.65 or num_vertices == 3:
    object_type = "Triangle"
elif extent >= 0.82 and aspect_ratio >= 0.78:
    object_type = "Square"
elif circularity >= 0.85 or (0.68 <= extent < 0.82 and aspect_ratio >= 0.80):
    object_type = "Circle"
else:
    object_type = "Rectangle"
```

Copy the indentation exactly. The lines inside each branch sit four spaces further right than the `if` that owns them. Python has no braces, so that indentation is the only thing telling it where a block starts and stops. If you see `IndentationError`, you have mixed tabs and spaces. In VS Code, click `Spaces: 4` in the status bar to switch the file to spaces, and stay there.

STEP 13

Two TODOs left, both down in `main()`. This one is about which camera to open.

```
cap_num = find_working_camera()
if cap_num is None:
    print("Error: No working camera stream found.")
    return
```

`find_working_camera()` tries indexes 0 to 4 and returns the first one that produces a picture. Linux does not promise a camera is always index 0 — plugging one in can push the other to index 1.

The `is None` test asks whether the function found nothing at all, and `return` leaves `main()` immediately rather than carrying on with a camera that does not exist.

STEP 14

Open the camera and ask for a sensible picture size.

```
cap = cv2.VideoCapture(cap_num, cv2.CAP_V4L2)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

`CAP_V4L2` is the Video4Linux2 backend, which is what the arm's Linux system uses. The two `set` lines request 640×480.

Note the wording: you are **asking**, not telling. Some cameras ignore the request, so if the window looks wrong, print

`cap.get(cv2.CAP_PROP_FRAME_WIDTH)` and find out what you actually got.

Both lines live inside `main()` rather than at the top of the file, because the camera is opened as late as possible and released as early as possible.

STEP 15

Send the file across and run it one more time. Then place one shape at a time on the plate and watch the label.

```
scp M2-P1-Starter.py er@192.168.1.149:Documents
```

```
python M2-P1-Starter.py
```

- ☐ **Triangle** — should print `verts: 3` and an extent near 0.50
- ☐ **Square** — extent near 1.00 and aspect ratio near 1.00
- ☐ **Circle** — circularity near 1.00 and extent near 0.79
- ☐ **Rectangle** — the long thin one; circularity 0.70 or lower, aspect ratio near 0.50

- *Test each shape at least three times, and move it around the plate. A detector that only works in the middle of the frame is not finished.*

READING THE OUTPUT

The line printed under the windows is your diagnostic. It looks like this:

```
[Detected] Square | Extent: 0.99 | Circ: 0.78 | Verts: 4
```

- **The word before the first pipe is the answer** — the only part the arm would ever act on.
 - **Extent, Circ and Verts are the evidence.** When the answer is wrong, these three tell you which measurement to distrust.
 - If `Verts` says 3 but the answer is not Triangle, then the shape is a poor drawing rather than your code being broken.
- *Change one threshold, retest, and write down what moved. Changing three numbers at once is how a whole afternoon disappears.*



EXTENSION 1

MAKE THE ARM SORT BY SHAPE

THE BRIEF

You have a program that names a shape. A sorting program needs one more thing: somewhere for each shape to go.

Add that, and the machine you have been building all module becomes a shape sorter. Then change what 'sort' means, because recognising four shapes is the easy half.

- *This challenge has no single correct answer. It has a working machine and a written explanation, and you need both.*

YOUR MISSION: SORT BY SHAPE

Record every shape's home position, then move it.

- 1 Move the arm to each bin**
Pick four positions on the plate. Each one is a list of six joint angles plus a gripper value, exactly like `home_pos`. Send the arm there when a shape is recognised.
- 2 Prove all four still work**
Twelve placements, three of each shape, every one identified correctly and sent to the right bin. If tuning a threshold fixed one shape and broke another, you have moved the problem rather than solved it.
- 3 Change the rule**
One shape per bin is the boring version. Try size instead of identity: big shapes into one bin, small into another. Or send triangles and squares to quality control and let only circles and rectangles through. Write your rule as plain sentences before you code it.
- 4 Hand it over**
Write the handover page: your thresholds, your sorting rule in plain language, and one thing you would fix with another hour.

● *Implement your rule by changing the logic, not by re-recording coordinates. Re-teaching the arm where the bins are means you solved the wrong problem.*

YOUR DATA LOG

Write down what you measured before you tune anything. If a threshold is about to change, you need to know what it was.

Shape	Extent	Circ	Aspect	Verts	My values
Triangle					
Square					
Circle					
Rectangle					

Then write your sorting rule as a set of conditions before you code it:

```
IF the shape is _____ THEN send it to _____  
IF the shape is _____ THEN send it to _____  
OTHERWISE _____
```

● *A rule you cannot say out loud is a rule you have not decided yet.*

DEFINITION OF DONE

- ☐ All four shapes detected correctly from three different places on the plate
- ☐ Every threshold change recorded in your data log, before and after
- ☐ Twelve placements sorted correctly into four bins
- ☐ A sorting rule of your own design, implemented in the **logic** rather than the coordinates
- ☐ Handover page written, and understood by your facilitator from the page alone

GO FURTHER

- Add a fifth shape. A cross or a five-pointed star needs a new test and a new place in the chain — decide where it goes before you write it, and remember that order matters.
- Make the program refuse to guess. If a shape's measurements sit within a hair of two thresholds, leave it on the plate and print a warning instead of naming it.
- Give the operator a choice of sorting rule when the program starts, so one machine can do several jobs.
- Log every detection to a file with a timestamp. Run the sorter for an hour and you will find shapes you never tested for.

MISSION DEBRIEF

1. Why does a circle score about 0.79 on extent rather than 1.00?
2. The square test and the circle test both look at `aspect_ratio`. What would happen if you swapped them?
3. You changed a threshold to make one shape work. How would you know you had not broken another?
4. If the camera could see colour as well as shape, what would you sort by instead — and which would you trust more?



steamoji™

MISSION

COMPLETE